

David Hill

SIMULINK-BASED CLOSED-LOOP MOTOR POSITION CONTROL ON ARDUINO: **DEVELOPMENT, TUNING AND PRACTICAL CONTROLLER COMPARISON**

CONTENTS

1. Project Aim
2. Hardware and Software Setup
3. Stage 1: Open-Loop Motor Control
4. Stage 2: Encoder Feedback and First Closed Loop
5. Stage 3: Tuned Practical Position-Hold Controller
6. Stage 4: Proportional Controller Comparison
7. Stage 5: Disturbance Rejection and Observations
8. Key Tuning Lessons
9. Conclusions and Next Steps

PROJECT AIM

The aim of this project was to develop a small embedded motor-control demonstrator in Simulink using Arduino hardware, PWM motor drive and quadrature encoder feedback. The project was intended to build practical experience of closed loop control development, hardware interfacing, model deployment and controller tuning on a real physical system.

The work progressed from simple open-loop motor actuation to encoder-based position feedback, then to a practical closed-loop position-hold controller. A further aim was to compare a hand-built practical controller against a cleaner proportional control structure and to understand how real hardware effects such as friction, backlash, breakaway torque and quantised feedback influence behaviour.

HARDWARE AND SOFTWARE SETUP

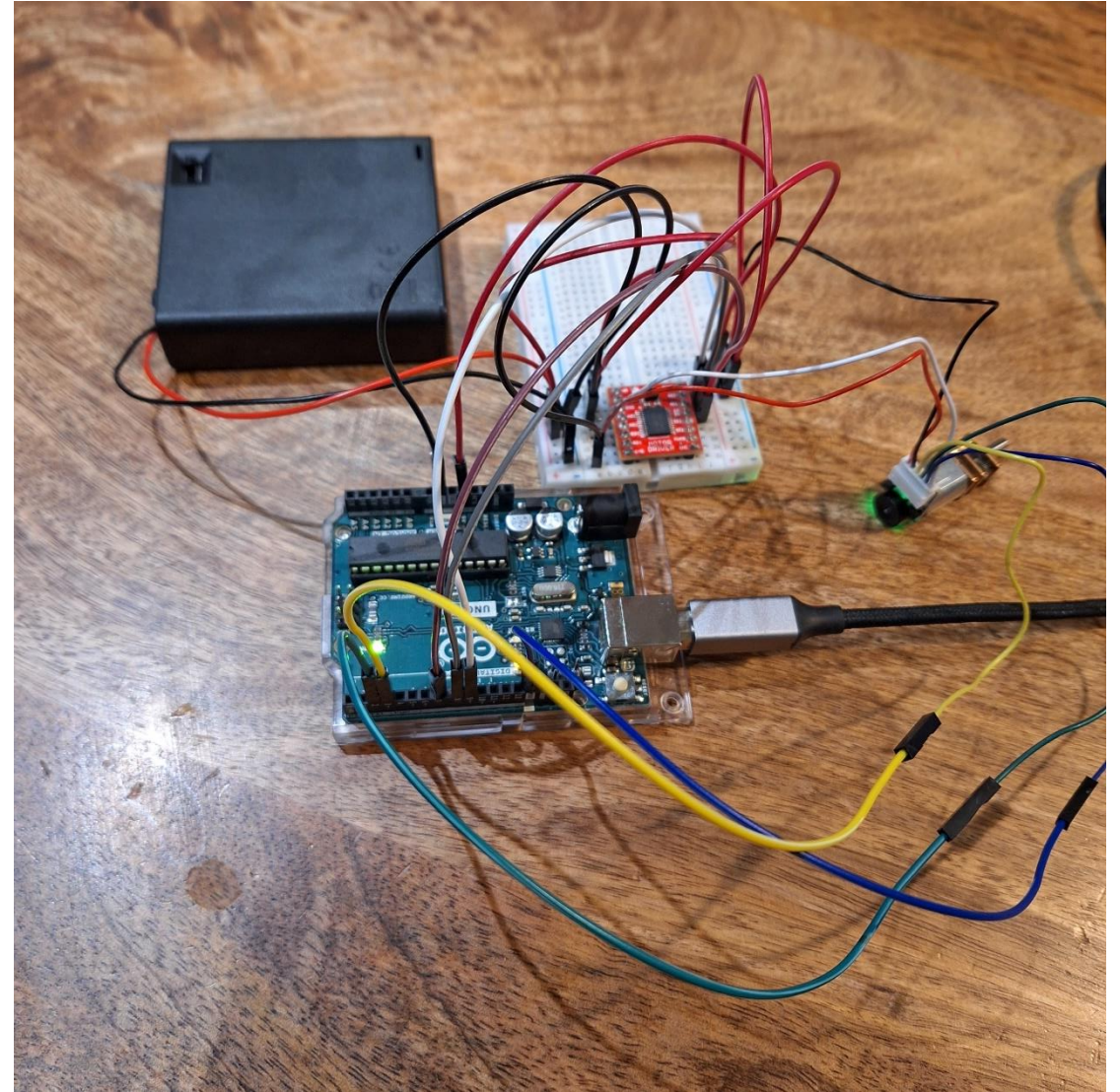
Hardware

- Arduino Uno R3 for Simulink-based control and encoder support
- Arduino Uno R4 Minima used earlier for initial open-loop Simulink motor control
- Micro metal geared DC motor with quadrature encoder
- TB6612FN motor driver
- External battery pack for motor drive
- Breadboard and jumper wiring for rapid prototyping

Software

- MATLAB and Simulink
- Simulink support package for Arduino hardware
- Simulink dashboard controls for live target and command adjustment

HARDWARE AND SOFTWARE SETUP



STAGE 1: OPEN-LOOP MOTOR CONTROL

Objective

To prove basic motor actuation from Simulink by controlling speed and direction through the motor driver.

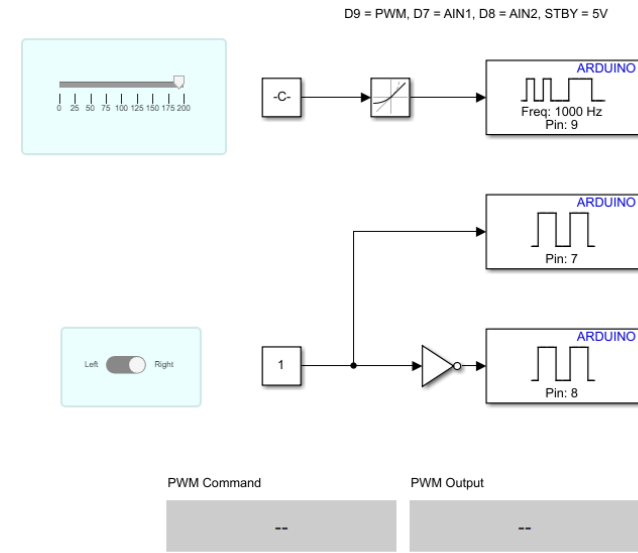
Implementation

- PWM output from Simulink to D9
- Direction control from Simulink to D7 and D8
- TB6612FNG used as the motor driver interface
- Dashboard slider used for live PWM adjustment
- Dashboard rocker switch used for live direction control

Outcome

- Motor speed was controlled successfully from Simulink
- Motor direction could be reversed live from the dashboard
- Proven signal chain from model to hardware.

STAGE 1: OPEN-LOOP MOTOR CONTROL



STAGE 1: OPEN-LOOP MOTOR CONTROL

Key Learning

- Simulink dashboard controls are parameter bound rather than directly wired like standard signal blocks.
- Basic embedded hardware deployment from Simulink was verified

STAGE 2: ENCODER FEEDBACK AND FIRST CLOSED- LOOP

Objective

To add position feedback using the motor encoder and build the first closed-loop position control structure in Simulink

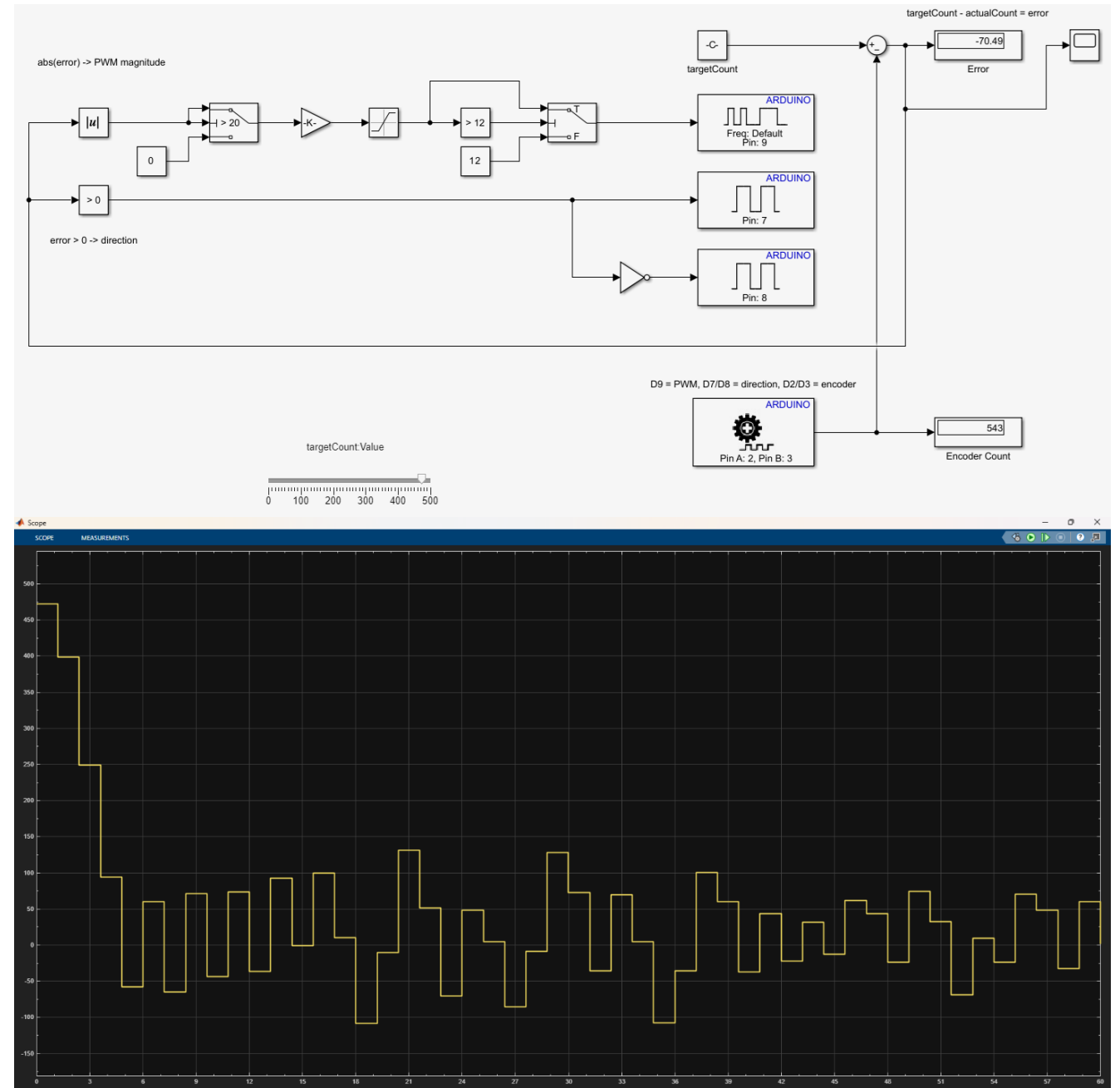
Implementation

- Quadrature encoder connected to D2 and D3
- Encoder count read into Simulink and displayed live
- Open-loop direction testing used to confirm sign convention
- Direction determined from the sign of the error
- PWM demand based on the magnitude of the error

Outcome

- Encoder feedback was successfully integrated into Simulink
- Motor driven while encoder count updated live
- First real closed-loop position controller achieved
- Project moved to command-and-feedback-control

STAGE 2: ENCODER FEEDBACK AND FIRST CLOSED- LOOP



STAGE 2: ENCODER FEEDBACK AND FIRST CLOSED- LOOP

Key Learning

- Correct sign mapping had to be proven experimentally before tuning.
- A real closed-loop system on small hardware is strongly affected by friction, backlash, breakaway torque and encoder quantisation.

STAGE 3: TUNED PRACTICAL POSITION- HOLD CONTROLLER

Objective

To improve the first closed-loop controller into a practical position-hold system that could move to a target and remain near it on real hardware.

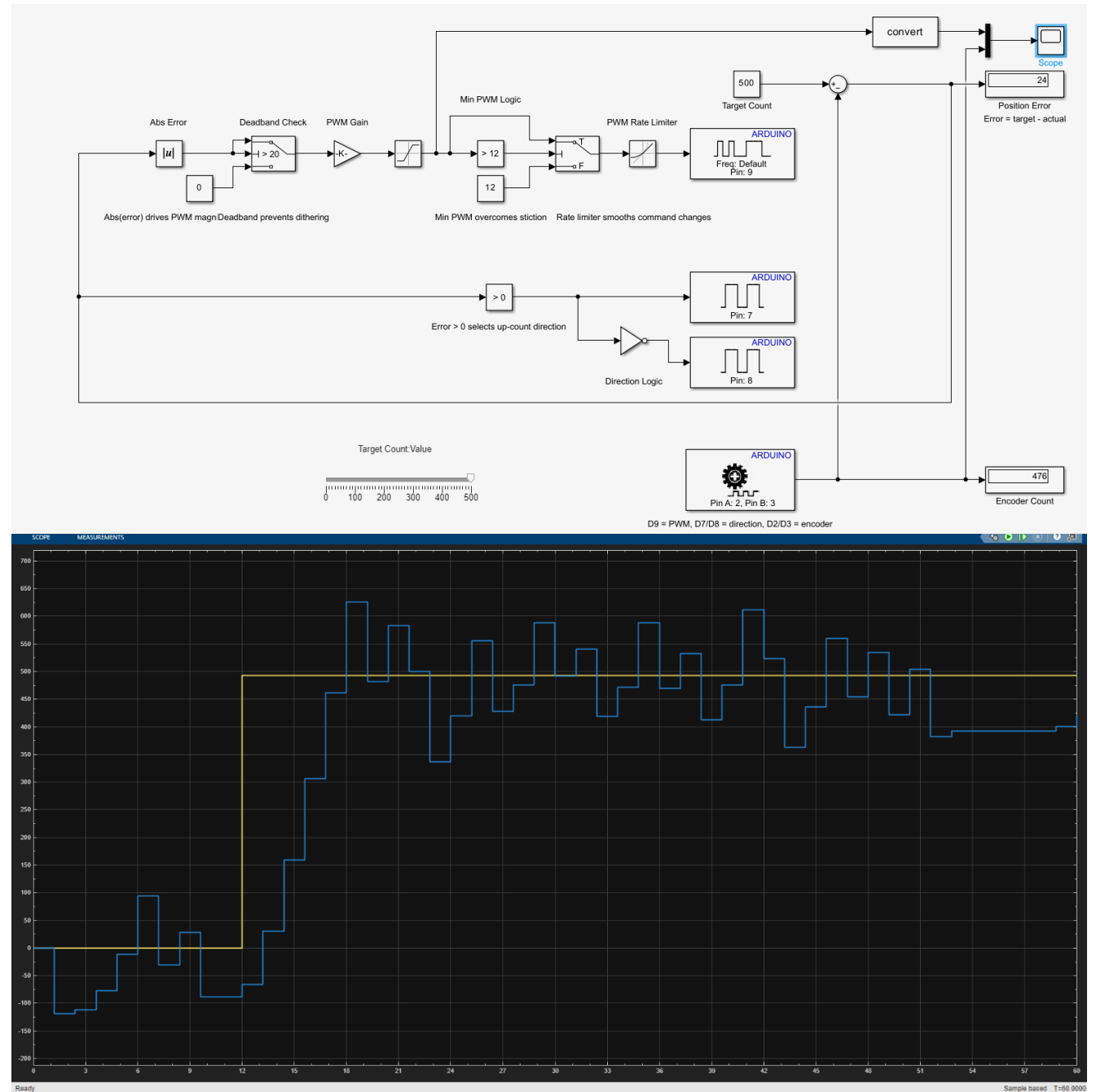
Implementation

- $\text{Error} = \text{targetCount} - \text{actualCount}$
- If $\text{error} > 0$ command up count direction
- If $\text{error} < 0$ command down count direction
- If error is inside deadband then stop
- If motion required but demand is too small then apply minimum PWM

Outcome

- The controller could move toward a commanded target and hold position reasonably well
- Live target changes could be followed in Simulink
- The model was much more usable on real hardware than the earlier controller.

STAGE 3: TUNED PRACTICAL POSITION- HOLD CONTROLLER



STAGE 3: TUNED PRACTICAL POSITION- HOLD CONTROLLER

Key Learning

- Deadband had to operate raw position error counts.
- Minimum PWM required to overcome stiction.
- Breakaway threshold for this setup was approx. PWM = 12
- Too much min PWM caused rocking, too little caused no movement at all.

STAGE 4: PROPORTIONAL CONTROLLER COMPARISON

Objective

To compare the practical Stage 3 controller against a cleaner proportional control structure in Simulink

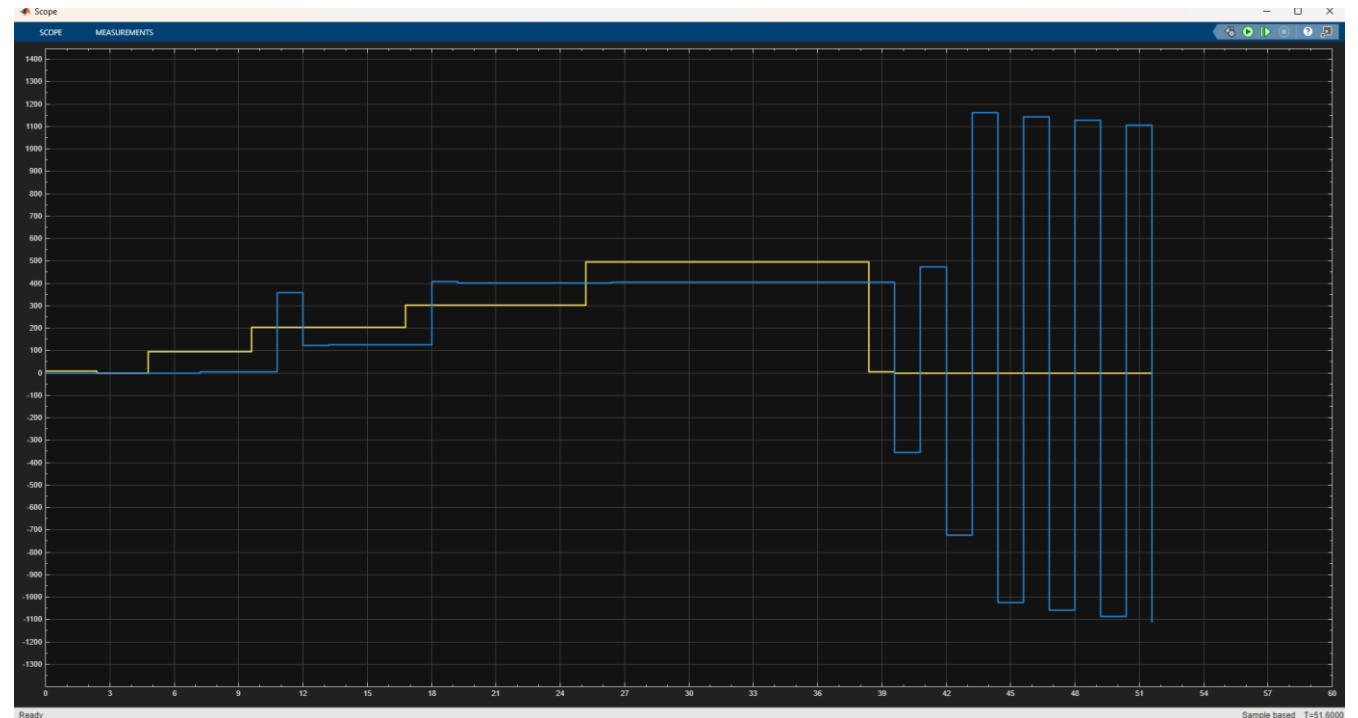
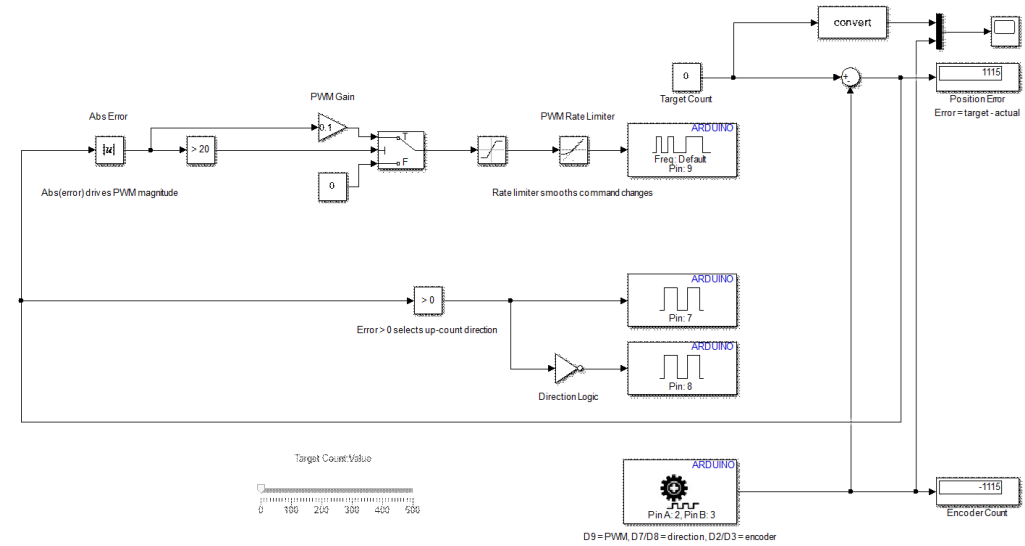
Comparison Approach

- Three forms were explored
 - Stage 4A: pure proportional controller
 - Stage 4B: proportional controller with deadband and rate limiter
 - Stage 4C: further command limiting to improve large step behaviour

Outcome

- The proportional controller was simpler and cleaner than the Stage 3 logic...
- ...but less robust on real motor/gearbox system
- Small target changes could be handled acceptably
- Larger target changes led to oscillation and repeated reversal.
- Reduced command limiting did not remove basic instability fully

STAGE 4: PROPORTIONAL CONTROLLER COMPARISON



STAGE 4: PROPORTIONAL CONTROLLER COMPARISON

Key Learning

Cleaner controller architecture doesn't necessarily perform better on real hardware. For this setup, backlash, stiction, breakaway torque and encoder quantisation strongly influenced the response and Stage 3 controller remained more usable.

STAGE 5: DISTURBANCE REJECTION AND OBSERVATIONS

Objective

To observe how the controller behaved when a target position changed and to assess how well it could hold and recover on real hardware.

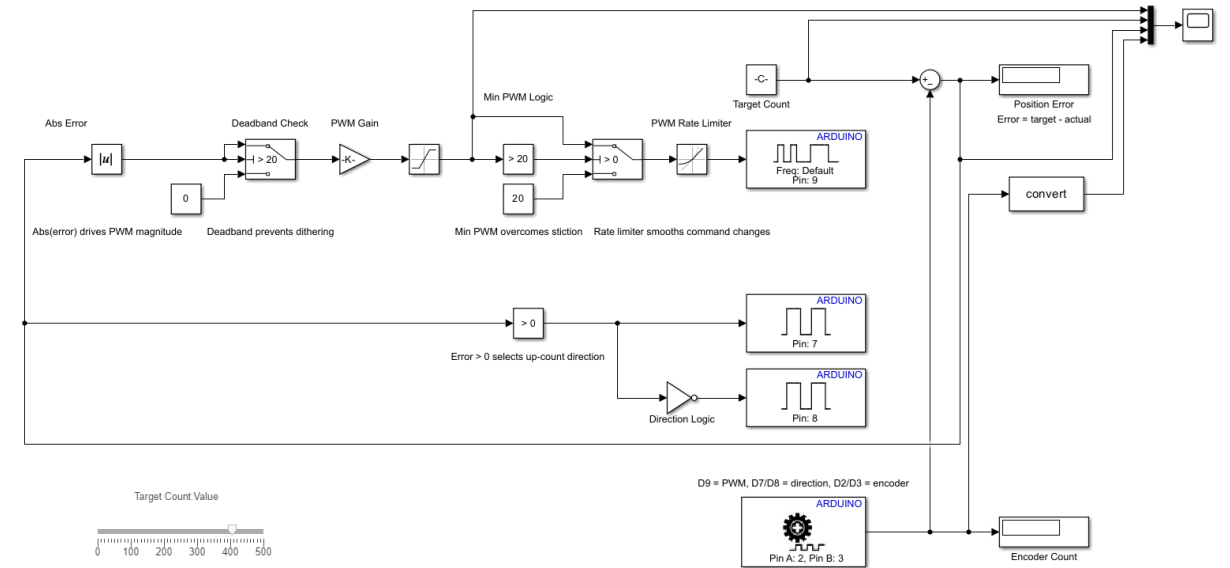
Implementation

- Target count changed during live operation
- Response and recovery behaviour using Simulink scopes
- Signals viewed together included:
 - Target Count
 - Encoder Count
 - Position Error
 - PWM Command

Outcome

- The controller followed target changes and produced the expected corrective response.
- Encoder tracked the commanded position – but not perfectly.
- Position error reduced after target changes, but still some residual oscillation rather than perfectly quiet hold.

STAGE 5: DISTURBANCE REJECTION AND OBSERVATIONS



STAGE 5: DISTURBANCE REJECTION AND OBSERVATIONS

Key Learning

- Friction, backlash, breakaway torque and encoder quantisation all affected the final hold quality.
- Stage 3 remains the most practical controller as it combines deadband, minimum PWM and rate limiting.

KEY TUNING LESSONS 1

Sign Convention Must be Proven

Before tuning, the motor direction and encoder count direction had to be proven.

D7=1, D8=0 increased count

D7=0, D8=1 decreased count

Deadband Must Act on Position Error

The controller behaved more sensibly when deadband was based on raw position error counts.

Overcome Breakaway Friction

Without a minimum drive floor, the controller could command motion but fail to move the motor. The practical threshold for this system was found to be PWM = 12.

KEY TUNING LESSONS 2

High Minimum PWM Causes Oscillation

If the minimum PWM is too high, each correction step became too aggressive and caused overshoot and repeated reversal around the setpoint.

Real Hardware Dominated Behaviour

The response was heavily influenced by friction, backlash, breakaway torque and encoder quantisation. The non-ideal effects shaped the tuning far more than a simple model would suggest.

Cleaner Control Laws Are Not Always Best

The proportional controller in Stage 4 was clearer structurally, but the more practical Stage 3 controller gave better real hardware performance as it handled the non-linear behaviour of the plant better.

CONCLUSIONS

The most valuable outcome was not simply making the motor move, but understanding how controller structure, tuning choices and plant non-linearities interact on a real embedded control system.

CONCLUSIONS

A complete Simulink based motor-control demonstrator was developed using Arduino hardware. The project progressed from:

- Open-loop actuation
- to encoder feedback
- to closed-loop position control
- to controller comparison and tuning

The most practically successful controller was the Stage 3 position-hold controller that combined:

- Deadband
- Minimum PWM
- Rate Limiting

The Stage 4 proportional controller was cleaner structurally, but less robust on the motor/gearbox system.

The project showed clearly that real hardware effects such as friction, backlash, breakaway torque and encoder quantisation strongly affect control behaviour.